

Concurrent Task Dispatcher in Rust - Design Report

For this project, I created a concurrent task dispatcher in Rust that simulates how a scheduler works in a multi-threaded environment. The main goal was to generate tasks, place them into queues, and distribute them across multiple worker threads while tracking system performance and behavior. I mainly wanted to better understand concurrency in Rust and see how different scheduling methods affect performance.

The program simulates both CPU-bound and IO-bound tasks. Instead of creating a real operating system scheduler, the project focuses on the overall idea of task scheduling through worker pools, queues, dispatching logic, and monitoring. I implemented two scheduling approaches and compared their performance under different workloads.

The first scheduling method uses FIFO scheduling, where tasks are processed in the same order they arrive. The second scheduling method is an optimized version that separates CPU and IO tasks into different queues. The purpose of this was to reduce delays caused by CPU-heavy tasks and improve responsiveness for shorter IO tasks.

The system is organized around task generation, queue management, worker threads, scheduling logic, and monitoring. The program begins by generating a large number of tasks using a fixed random seed so the same workloads can be reproduced consistently. Each task contains an ID, task type, duration, and CPU cost. Some tasks represent CPU-heavy work while others represent IO-heavy work.

The worker pool contains eight worker threads. Each worker continuously checks the queues for available tasks, processes them, records metrics, and continues until all tasks are completed. Thread sleeping is used to simulate execution time for the tasks.

The FIFO scheduler uses one shared queue where tasks are processed strictly in arrival order. The optimized scheduler uses separate queues for CPU tasks and IO tasks. This allows shorter IO tasks to finish earlier instead of waiting behind longer CPU-heavy tasks.

I also added a monitor thread to track information such as CPU usage, active workers, queue length, and elapsed runtime while the simulation is running. The collected information is saved into a CSV file for later viewing.

One of the main parts of this project involved safely sharing data between multiple threads. Since several worker threads access the queues and results at the same time, synchronization was necessary to avoid race conditions and data corruption.

To handle this, I used Arc, Mutex, and Atomic variables throughout the project. Arc allowed multiple threads to share ownership of the same data structures. Mutex ensured that only one thread could modify shared resources such as queues or result vectors at a time. Atomic variables were useful for tracking values like CPU usage and active worker counts without needing full mutex locks.

One issue I ran into during development was that the optimized scheduler originally showed almost no difference compared to FIFO scheduling. After testing the program, I realized this happened because every task had the same execution duration. To make the behavior more realistic, I adjusted the task durations so CPU tasks took longer than IO tasks. After making this change, the optimized scheduler showed lower average wait times and better responsiveness.

Another important part of the project was making sure the program shut down correctly. All worker threads needed to terminate once all tasks were completed, and the monitor thread also needed to stop cleanly. This was handled by joining all threads at the end of execution.

Two experiments were performed during testing. The first experiment used a balanced workload with mostly IO tasks and fewer CPU tasks. Under this workload, the optimized scheduler reduced average wait times because IO tasks were processed more efficiently.

The second experiment used a CPU-heavy workload designed to put more stress on the scheduler. Even under heavier CPU pressure, the optimized scheduler still produced lower average wait times compared to FIFO scheduling.

The program collected several performance metrics during execution, including total runtime, makespan, average wait time, average turnaround time, CPU usage, active worker counts, and completed task totals. The experiments showed that separating CPU and IO tasks improved responsiveness, especially when CPU-heavy workloads became more dominant.

Overall, this project helped me better understand concurrency, synchronization, worker pools, queues, and scheduling strategies in Rust. The FIFO scheduler was easier to implement, but the optimized scheduler performed better under mixed workloads because it separated CPU and IO tasks into different queues. Through this project, I gained more experience working with threads, shared state, synchronization tools, and performance analysis. I also developed a better understanding of how scheduling decisions influence system behavior in concurrent systems.